



Dokumentacja Techniczna

Projekt kasyna internetowego "Absolute Casino"

Autorzy: Mateusz Kroplewski, Aleksander Olkowicz, Paweł
Wójcik, Kacper Araszkiewicz

Data: 24 marca 2025

Spis treści

1	Opis problemu	2
1.1	Porównanie dostępnych rozwiązań	2
1.2	Możliwości zastosowania praktycznego	2
1.3	Aspekt badawczy projektu	2
2	Projekt i analiza	2
2.1	Aktorzy i przypadki użycia	2
2.2	Wymagania funkcjonalne	3
2.3	Diagramy klas	4
2.4	Diagram modelu danych	7
2.5	Projekt interfejsu użytkownika	7
3	Implementacja	7
3.1	Architektura rozwiązania	7
3.2	Użyte technologie	8
4	Testy	8
4.1	Scenariusz testowania	8
4.1.1	Backend	9
4.1.2	Frontend	9
4.2	Raport z testów	10
4.2.1	Testy jednostkowe Mummy	10
4.2.2	Testy jednostkowe Fruitogedon	10
4.2.3	Testy jednostkowe Blackjack	10
4.2.4	Testy jednostkowe Baccarat	10
4.2.5	Testy E2E	10
5	Wkład własny	10

1 Opis problemu

1.1 Porównanie dostępnych rozwiązań

1.2 Możliwości zastosowania praktycznego

1.3 Aspekt badawczy projektu

2 Projekt i analiza

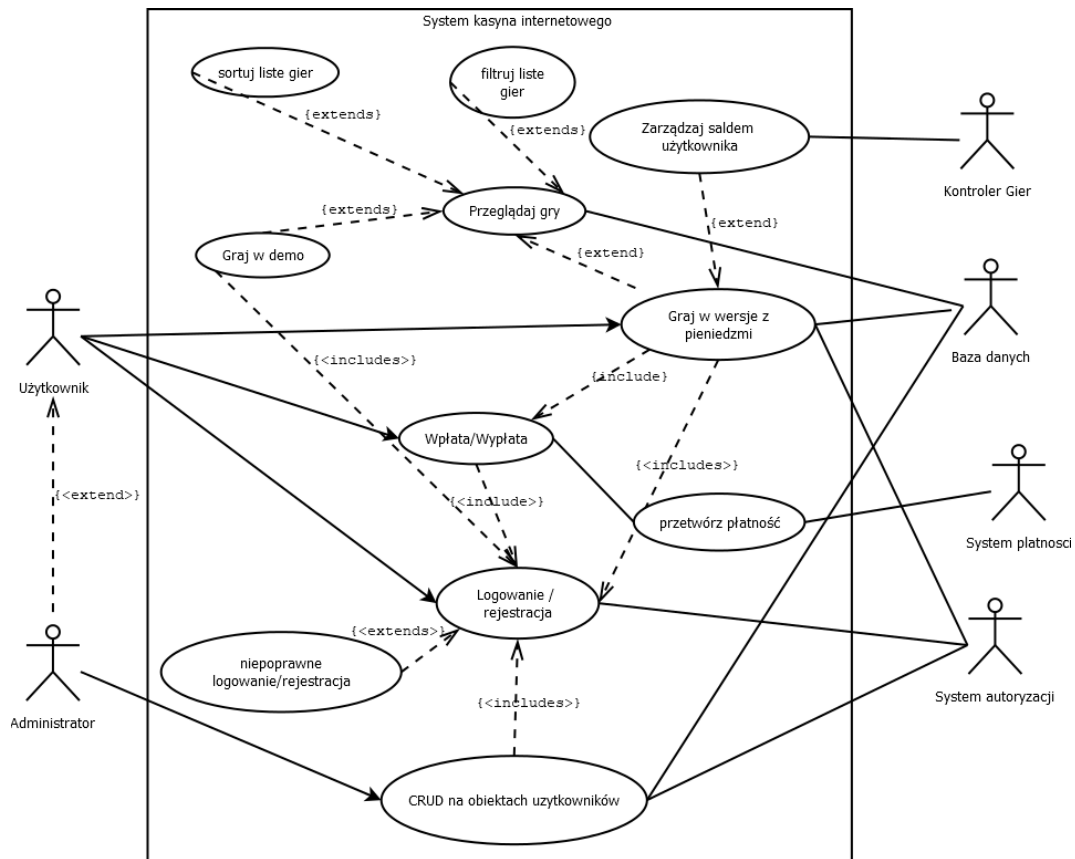
2.1 Aktorzy i przypadki użycia

Aktorzy

1. **Gracz** – użytkownik końcowy, który loguje się do systemu, wpłaca środki, gra w gry hazardowe i wypłaca wygrane.
2. **Administrator** – osoba zarządzająca systemem, moderująca użytkowników, monitorująca transakcje oraz ustawienia gier.
3. **System Płatności** – system obsługujący wpłaty i wypłaty środków.
4. **System bonusów** – moduł odpowiedzialny za przydzielanie bonusów do gier slotowych.
5. **System Losujący ([rng]RNG – Random Number Generator)** – moduł odpowiedzialny za losowe wyniki gier.

Przypadki Użycia

1. **Rejestracja nowego gracza** – Gracz zakłada konto, podając wymagane dane i potwierdzając tożsamość.
2. **Logowanie gracza** – Gracz loguje się do systemu za pomocą loginu i hasła.
3. **Wpłata środków** – Gracz dodaje środki na swoje konto poprzez system płatności.
4. **Wybór gry** – Gracz wybiera pomiędzy grami slotowymi a karcianymi.
5. **Rozgrywka** – Gracz uczestniczy w grze, system losuje wyniki.
6. **Odbiór wygranej** – Gracz otrzymuje wygrane środki na konto.
7. **Konfiguracja gier** – Administrator dostosowuje ustawienia gier.
8. **Blokowanie użytkownika** – Administrator może zablokować konto gracza w przypadku naruszenia regulaminu.



Rysunek 1: Diagram przypadków użycia

2.2 Wymagania funkcjonalne

1. Rejestracja i logowanie:

- Możliwość rejestracji nowych użytkowników.
- Weryfikacja tożsamości graczy za pomocą tokenu [jwt]JWT.

2. Obsługa wpłat i wypłat:

- Symulacja systemu wpłat.

3. Gry hazardowe:

- Implementacja gier slotowych i karcianych.
- Zastosowanie algorytmu RNG dla uczciwości losowania.
- Możliwość ustawiania limitów zakładów dla danej gry.
- System bonusów

4. Bezpieczeństwo i zgodność:

- Mechanizmy ochrony przed nadużyciami implementacji gier.

5. Zarządzanie użytkownikami:

- Możliwość blokowania kont naruszających regulamin.
- Historia transakcji i gier dostępna dla administratora.

6. Interfejs użytkownika:

- Responsywny interfejs dostępny na komputery i urządzenia mobilne.
- Czytelne statystyki wygranych i przegranych.

Wymagania niefunkcjonalne

1. Wydajność:

- Czas odpowiedzi systemu nie powinien przekraczać 1 sekundy.

2. Bezpieczeństwo:

- Szyfrowanie danych użytkowników.

3. Skalowalność:

- Możliwość dodawania nowych gier bez konieczności modyfikacji całego systemu.

4. Dostępność:

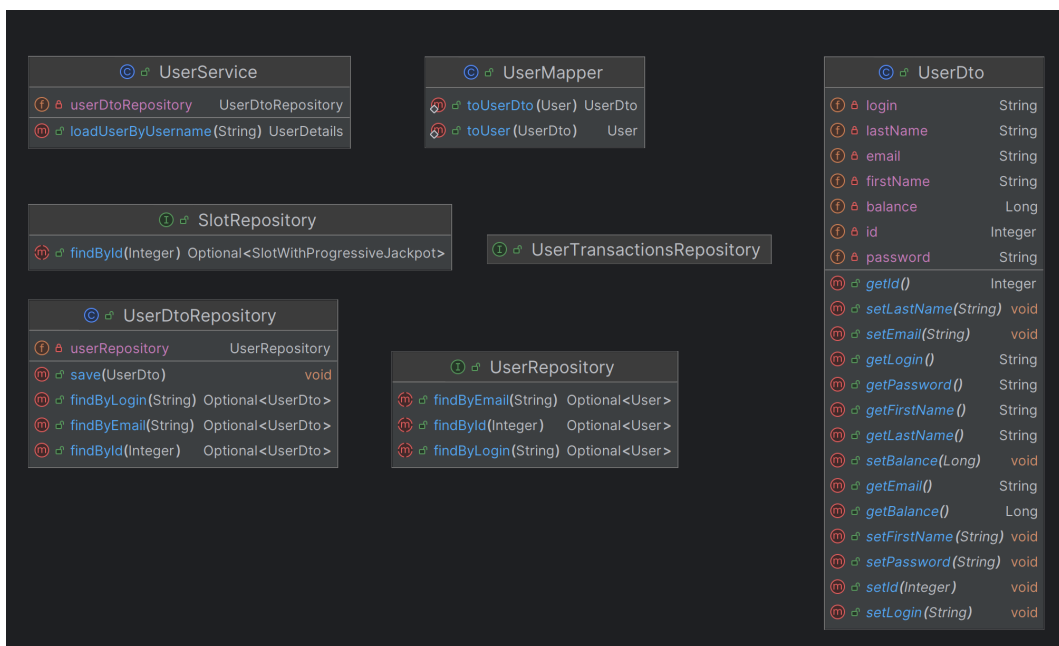
- System dostępny 24/7.
- Maksymalny czas niedostępności nie przekracza 5 minut miesięcznie.

2.3 Diagramy klas

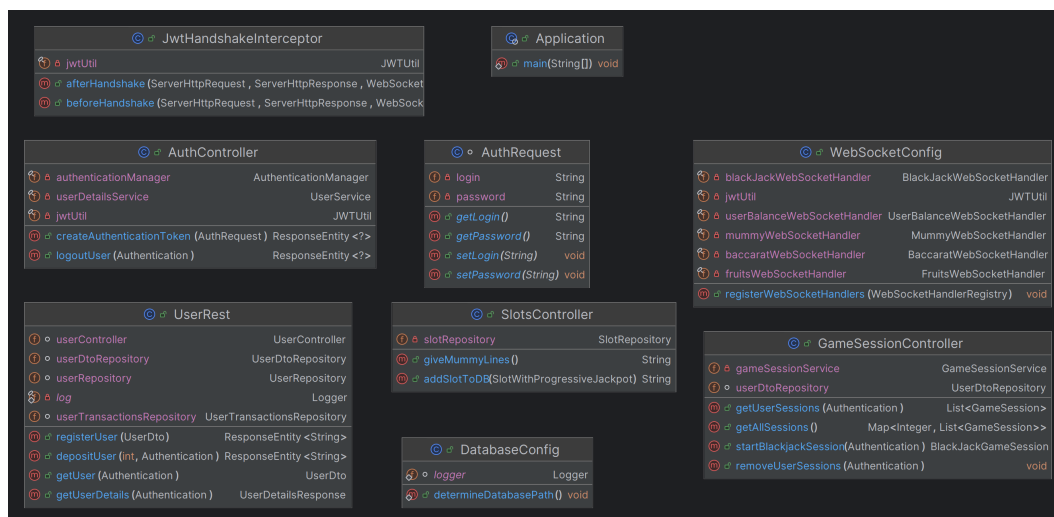
W tej sekcji przedstawiamy diagramy klas dla systemu gier. Poniżej znajdują się diagramy dla różnych aspektów systemu.



Rysunek 3: Diagram klas - Model



Rysunek 4: Diagram klas - Repository



Rysunek 5: Diagram klas - View

2.4 Diagram modelu danych

2.5 Projekt interfejsu użytkownika

3 Implementacja

3.1 Architektura rozwiązania

System "Absolute Casino" został zaprojektowany w oparciu o podejście wielowarstwowe, co pozwala na logiczne oddzielenie poszczególnych funkcjonalności. W skrócie, nasza architektura składa się z następujących elementów:

- **Backend:** Nasz serwer napisany jest w Javie przy użyciu Spring Boot, co umożliwia szybkie i elastyczne tworzenie aplikacji. Zaimplementowaliśmy własny mechanizm autentykacji, dzięki któremu mamy pełną kontrolę nad logowaniem i zarządzaniem sesjami użytkowników. Dodatkowo, backend obsługuje komunikację przez WebSockets – jeden kanał jest wykorzystywany do dynamicznych rozgrywek, a drugi do aktualizacji stanu konta.
- **Frontend:** Interfejs użytkownika zbudowany został w React. Dzięki wykorzystaniu bibliotek takich jak Tanstack Router, Pixi.js, Shadcn, Tailwind CSS oraz Reactspring, udało nam się stworzyć responsywną i atrakcyjną wizualnie aplikację. Umożliwia ona płynne przechodzenie między widokami, dynamiczne renderowanie gier oraz animacje, co jest kluczowe przy obsłudze interaktywnego środowiska kasyna.
- **Baza danych:** Podczas prac deweloperskich korzystamy z lekkiej bazy H2, co znacząco ułatwia testowanie i rozwój aplikacji. W produkcji planujemy użyć PostgreSQL – system ten zapewnia stabilność i skalowalność, które są niezbędne przy dużej liczbie użytkowników.
- **Komunikacja w czasie rzeczywistym:** Cała komunikacja między frontendem a backendem odbywa się przez WebSockets. To podejście umożliwia natychmiastowe przesyłanie wyników gier oraz szybką aktualizację stanu konta użytkownika.

3.2 Użyte technologie

W trakcie tworzenia "Absolute Casino" zdecydowaliśmy się na zestaw nowoczesnych narzędzi, które ułatwiają zarówno rozwój aplikacji, jak i utrzymanie jej wysokiej wydajności. Oto, z czego korzystamy:

- **Backend:**
 - **Java + Spring Boot:** Główny framework, który umożliwia tworzenie skalowalnych aplikacji.
 - **Własna autentykacja:** Implementacja mechanizmu logowania i zarządzania sesjami, dzięki której zabezpieczamy dostęp do aplikacji.
- **Frontend:**
 - **React:** Podstawa naszego interfejsu, która pozwala na tworzenie interaktywnych i dynamicznych widoków.
 - **Tanstack Router:** Ułatwia zarządzanie trasami w aplikacji, dzięki czemu przejścia między widokami są płynne.
 - **Pixi.js:** Biblioteka służąca do renderowania gier – gwarantuje wysoką wydajność i ładną grafikę.
 - **Shadcn:** Zestaw komponentów, który przyspiesza tworzenie spójnego i nowoczesnego interfejsu.
 - **Tailwind CSS:** Narzędzie do stylizacji, które pozwala szybko tworzyć responsywne i estetyczne strony.
 - **React-spring:** Umożliwia tworzenie efektownych animacji, dodając aplikacji dynamiki.
- **Baza danych:**
 - **H2:** Używana lokalnie podczas developmentu, dzięki czemu konfiguracja i testowanie są bardzo proste.
 - **PostgreSQL:** Docelowa baza danych w środowisku produkcyjnym, wybrana ze względu na niezawodność i skalowalność.
- **Komunikacja:**
 - **[websocket]WebSocket:** Stosujemy natywną obsługę WebSocketów, co pozwala na efektywną i szybką komunikację w czasie rzeczywistym. Wykorzystywane są dwa kanały – jeden do gier, drugi do aktualizacji stanu konta.

Podsumowując, nasz projekt "Absolute Casino" łączy solidne rozwiązania backendowe oparte na Spring Boot z nowoczesnym i dynamicznym frontendem w React. Wykorzystanie wydajnych baz danych oraz natywna obsługa WebSocketów gwarantują, że system jest zarówno elastyczny, jak i gotowy do pracy w środowisku produkcyjnym.

4 Testy

4.1 Scenariusz testowania

W niniejszej sekcji przedstawiono strategię testowania aplikacji, która została podzielona na dwa główne obszary: backend oraz frontend.

4.1.1 Backend

Testy backendu obejmują głównie `[unit_test]`testy jednostkowe (unit tests) skoncentrowane na logice poszczególnych gier, w tym:

- **Blackjack:** Testy walidują poprawność rozgrywki, obliczania wartości kart, stosowanie zasad gry oraz detekcję wygranej/przegranej.
- **Baccarat:** Testy sprawdzają algorytmy przydzielania punktów, wybór wyniku (gracz, bankier, remis) oraz zgodność z regułami gry.
- **Gry slotowe (Mummy i Fruitogedon):** Testy jednostkowe obejmują walidację algorytmów losujących oraz wyliczania wypłat. Szczególną uwagę poświęcono testowaniu mechanizmów odpowiedzialnych za:
 - `[rtp]`**RTP (Return to Player):** Parametr określający średni procent wypłat w stosunku do postawionych środków. Testy weryfikują, czy obliczenia RTP mieszczą się w oczekiwanych granicach i zapewniają uczciwość rozgrywki.
 - `[slot_volatility_variance]`**Volatility Index:** Wskaźnik zmienności, który definiuje częstotliwość i wielkość wygranych. Testy symulują długotrwałe rozgrywki, aby upewnić się, że rozkład wygranych odpowiada zaprojektowanemu poziomowi ryzyka.

W przypadku tego projektu testy jednostkowe backendu są kluczowe, ponieważ gwarantują, że oferowane gry są zgodne z zasadami i regulacjami. Gry karciane operują na sztywnych zasadach co ułatwia ich testowanie, oraz jasne określenie przypadków brzegowych.

Gry slotowe nie mają takich zasad co czyni testowanie ich trudniejszym. Jediną metodą pozwalającą określenie uczciwości gry jest metoda `[monte_carlo]`**Monte Carlo**, tj. rozegranie bardzo dużej ilości gier w celu zdeterminowania wskaźników takich jak RTP.

4.1.2 Frontend

Testowanie interfejsu użytkownika dzieli się na dwie grupy:

- **Testy jednostkowe:** Skupiają się na podstawowych komponentach, takich jak wyświetlanie wartości kart w grze Blackjack. Testy te weryfikują, czy dane są poprawnie renderowane, a logika wyświetlania spełnia oczekiwania projektowe.
- `[e2e]`**Testy E2E (End-to-End):** Stanowią większość testów frontendowych. Są to symulacje pełnych scenariuszy rozgrywki, które pozwalają na:
 - Symulację interakcji użytkownika z interfejsem – np. logowanie, wybór gry, podejmowanie zakładów.
 - Weryfikację przepływu rozgrywki w warunkach zbliżonych do rzeczywistych, gdzie system reaguje na decyzje gracza.
 - Testowanie komunikacji między frontendem a backendem, co umożliwia wykrycie ewentualnych problemów związanych z synchronizacją danych i odpowiedziami serwera.

Testy E2E są niezbędne, ponieważ pozwalają na sprawdzenie, czy wszystkie elementy systemu współpracują ze sobą poprawnie i czy aplikacja jako całość zachowuje się zgodnie z wymaganiami funkcjonalnymi. Jest to jedyny sposób na testowanie jak działa aplikacja z perspektywy użytkownika, czyli poprzez interakcję z interfejsem. Do zaimplementowania testów E2E użyliśmy frameworka `[playwright]`**Playwright**, który działa poprzez symulowania realnej przeglądarki. Framework oferuje zautomatyzowane interakcje z interfejsem

(np. kliknięcie w przycisk, wypełnienie formularza), lokalizowania elementów na stronie w celu walidacji wyświetlania poprawnych danych, oraz wiele innych funkcji.

By zagwarantować powtarzalność testów gier, oraz wyeliminować inherencyjną losowość algorytmów na backendzie używamy techniki mockingu. Oznacza to że zastępujemy komunikację z backendem, symulowanymi odpowiedziami, co pozwala nam na testowanie spreparowanych scenariuszy, w tym przypadków brzegowych.

4.2 Raport z testów

4.2.1 Testy jednostkowe Mummy

4.2.2 Testy jednostkowe Fruitogedon

4.2.3 Testy jednostkowe Blackjack

4.2.4 Testy jednostkowe Baccarat

4.2.5 Testy E2E

5 Wkład własny